

Langfuse Setup Guide for Multi-Agent AI Systems

What is Langfuse?

Langfuse is an open-source observability platform for LLM applications. It provides comprehensive monitoring and debugging capabilities for AI systems:

- **Tracing:** Track every LLM call, embedding, and agent action
- **Cost Monitoring:** See token usage and costs across all API calls
- **Latency Analysis:** Identify performance bottlenecks in your pipelines
- **Debugging:** Inspect inputs, outputs, and errors for each step
- **Session Tracking:** Group related traces into logical sessions

Quick Setup (5 minutes)

1. Create a Langfuse Account

Go to cloud.langfuse.com, sign up with GitHub or email, and create a new project (e.g., 'contract-intelligence').

2. Get Your API Keys

From your Langfuse project dashboard, copy your Secret Key (starts with sk-lf-) and Public Key (starts with pk-lf-).

3. Configure Environment Variables

Add the following to your .env file:

```
LANGFUSE_SECRET_KEY=sk-lf-your-secret-key
LANGFUSE_PUBLIC_KEY=pk-lf-your-public-key
LANGFUSE_HOST=https://cloud.langfuse.com
```

4. Verify Connection

```
from langfuse import Langfuse

langfuse = Langfuse()

langfuse.auth_check() # Returns True if configured correctly
```

Integration Patterns

Direct Tracing (OpenAI)

Create traces and log generations directly:

```
trace = langfuse.trace(name='my-analysis', session_id='session-001')
generation = trace.generation(name='gpt-call', model='gpt-4o-mini', input='...')
response = openai_client.chat.completions.create(...)
generation.end(output=response.choices[0].message.content)
```

LangChain Integration

Use the callback handler for automatic tracing:

```
from langfuse.callback import CallbackHandler as LangfuseCallbackHandler

handler = LangfuseCallbackHandler(session_id='session-001', trace_name='analysis')

llm = ChatOpenAI(model='gpt-4o-mini', callbacks=[handler])
```

Key Concepts

- Trace: Top-level container for a complete operation
- Span: A step within a trace (e.g., 'preprocessing')
- Generation: An LLM call with model, tokens, and latency
- Session: Groups related traces (e.g., one user conversation)
- Tags: Labels for filtering (e.g., ['production', 'contract'])

Best Practices

- Use Session IDs: Group related operations with consistent session_id
- Add Tags: Tag traces for easy filtering (['agent', 'risk-analysis'])
- Flush on Exit: Always call langfuse.flush() before script ends
- Name Traces Clearly: Use descriptive names like 'contract-risk-agent'

Troubleshooting

- Auth fails: Verify keys in .env, check for typos
- No traces appear: Call langfuse.flush() before exiting
- Missing data: Ensure generation.end() is called after each LLM call

Resources

Documentation: langfuse.com/docs | GitHub: github.com/langfuse/langfuse | Python SDK: github.com/langfuse/langfuse-python